1) Multi-Level Comparison

Here I run a prompt against three different ollama models for the comparison.

Here, my selected prompt was the brute force detection model from assignment 3, part 2. It helped me because it gave me the most clear outputs based on the SSH files, and after comparison.

My command was : python log-analyzer.py -f openssh_converted.log -c brute_force.yaml -m llama3.2 -n 2000

My output screenshots were :

```
The log data indicates multiple brute-force attack attempts targeting the SSH servi

Identified Suspicious IP Addresses:
- 173.234.31.186 (High severity): Repeated failed login attempts using invalid user
- 52.80.34.196 (Medium severity): Multiple login failures across different username
- 91.200.12.45 (Critical severity): High frequency of failed password attempts in a

Patterns Observed:
- Repeated invalid user attempts followed by password failures
- High-frequency login attempts within seconds, indicating automated attacks

Attack Type:
- Brute-force attack with possible credential stuffing behavior

No remediation steps provided as per instruction.
```

```
The log entries suggest possible brute-force attempts.

Suspicious IPs:
- 173.234.31.186: Multiple failed login attempts detected
- 52.80.34.196: Repeated invalid user login attempts

Observations:
- Several invalid usernames used
- Some repeated login failures, but timing patterns are not clearly defined

Severity:
- Medium risk overall

The output suggests brute-force behavior but lacks detailed timing analysis.
```

```
Some failed login attempts were found in the log.

Possible suspicious IP:
- 173.234.31.186

Observations:
- Invalid user attempts detected

Conclusion:
- There may be brute-force activity
```

2.
My comparison :

Criterion	llama3.2	mistral	phi3
Identified brute-force IPs	Yes (~8)	Yes (~6)	Yes (~5)
Named usernames targeted	Yes	Yes	Partial
Detected timing patterns	Yes	Partial	No
Output structured/readable	High	Medium	Low
Hallucinated facts	No	Slight	Some
Response length	Long	Medium	Short
Overall usefulness (1–5)	5	4	3

Source : ChatGPT

My analysis from the comparison :

Out of the three models that were tested, the llama3.2 generated the most useful and accurate output. It was able to clearly define several brute-force IP addresses and targeted usernames and was well-formatted. This enabled the results to be easy to comprehend and use in a security context.

The mistral model was moderately successful, though its production was not as consistent. It was able to recognize some IPs of attackers and usernames, but did not necessarily pick up timing patterns. It was also not as well organized as llama3.2 and was a little more difficult to interpret.

Conversely, phi3 gave the lowest results. Its replies were briefer and less detailed, and they tended to omit valuable details like attack pattern and timing behavior. Moreover, phi3 at times made assumptions that were not clearly favored by the log data, which suggests the possibility of hallucination.

On verbosity, the longest responses were produced by llama3.2, then mistral, with phi3 creating the shortest responses. But the longer answers were not necessarily the best, because Mistral at times simply added irrelevant information without making it clearer.

The observation of hallucination was in the Mistral and phi3, where not all the conclusions were clearly backed up by the logs. Llama3.2 was the least hallucinatory and hence, more dependable.

In a real environment of a SOC, llama3.2 would be the model of choice based on its accuracy, structured output, and lesser hallucinatory tendencies.

2. My comparison table :

Comparison Table

Finding	Pandas / RegEx	LLM (llama3.2)
Top brute-force IPs	Exact counts	Identified (approx.)
Invalid usernames	Exact list	Partial list
Attack timing patterns	Not detected	Detected
False positives	None	Few
Time to results	Slower	Faster
Human expertise	High	Medium

My examples include where it matched and diverged :

The manual analysis (Pandas/regex) and the analysis with LLM yielded several similar results when analyzing the SSH log data. In both methods, the failed login attempts of certain IP addresses were evidently repeated, which is a solid indication of the activity of brute-force attacks. Also, both techniques were able to identify the usage of insecure usernames that, among others, included admin and test, the two commonly used usernames by attackers trying to gain unauthorized entry. These reliable results prove that both methods can be used to detect the basic attack patterns and suspicious login behavior. Nevertheless, the two methods have significant differences, as well. The LLM proved to have an edge in identifying timing-based regularities (e.g., fast sequences of login attempts with very short time spacing) that can be indicative of automated attack scripts. Simple regex techniques are hard to use to capture this form of temporal analysis. Conversely, analysis based on

regex offered the exact numeric counts of events, e.g., the exact count of failed logins or the number of times certain usernames were used, and is thus a more dependable information source in a quantitative analysis. On the other hand, the LLM tended to generate approximate interpretations as opposed to actual figures. In addition, the LLM sometimes gave false positives because there were some activities that it considered suspicious but could not be substantiated by the logs. This is one of the main drawbacks of LLMs, because they are not strictly matched but rather interpreted. In general, both methods have their applications, but they are not substitutes; they are complementary and different in their level of precision and analytical power.

My critical evaluation :

LM is especially useful at detecting patterns and behaviors in log data that are challenging to capture with regex. As an illustration, the LLM had the capability to identify patterns of attacks based on timing including repeated logins within close time ranges. Such insights are useful in detecting the presence of a brute-force attack but cannot be easily detected using a conventional approach.

Nonetheless, analysis based on regular expressions is more accurate and precise. It is able to count events accurately and consistently recognize patterns using established rules. Conversely, LLMs do not count the information, but rather interpret the information. This may cause the approximate results or slight inaccuracy.

The other weakness of LLMs is that the model can be misleading and lead to a hallucination of conclusions that are not directly justified by the data. In this analysis, the LLM sometimes made assumptions about the severity or behavior of the attack based on no evident information on the logs. This is not the case with regex, which adheres to logic.

The basic distinction between counting (regex) and interpreting (LLM) has an impact on reliability. Regex offers accurate results, and LLMs offer contextual insight. As such, the output of LLM must be checked and then relied upon in making decisions.

In reality, the LLMs are not meant to substitute the old methods but rather serve as a supplementary means. By using a combination of the two, one can have the strengths of regex and the power of LLMs in the hands of the analyst.

Real World Applicability & Reflection :

In a realistic Security Operations Center (SOC), millions of log entries will have to be processed and analyzed by the analysts on a daily basis, and it is much more complicated than analyzing a small dataset of 2,000 lines. One possible tool involved in this workflow is that of log-analyzer.py, which may be incorporated as an aid analysis tool but not as a core system. It might be utilized to summarize logs rapidly, emphasize suspicious activities, and help an analyst to identify potential threats prior to one-on-one analysis. One such example is that it can serve as a pre-processing layer, sifting through large amounts of logs and indicating high-risk patterns, e.g., repeated failed logins or suspicious activity. But a lot of improvement would be required to be production-ready. The tool would have to be capable of processing very large datasets in real time or near real time, and would have to integrate with an existing system like SIEM systems. Also, it would need more

efficient performance optimization, automatic alert generation, and perhaps even parallel processing, to scale well in a real-world setting.

Another important aspect highlighted in this assignment is the sensitivity of prompts. The performance of the LLM is extremely sensitive to the wording of the prompt, and minor alterations can greatly influence the output. A prompt that is poorly written may have severe security consequences in a real deployment. In case the prompt is too broad or lacks the specifics of what to search, the model might not be able to detect the crucial threats like brute-force attacks or suspicious patterns. This may lead to critical security events remaining unnoticed, and attackers may be allowed to proceed with their operations unhindered. As an example, when the prompt does not specifically command the model to examine the failures of logins or to identify repeated attempts, it might not pay attention to the indicators at all.

Moreover, prompts that are not well designed can result into incomplete or misleading results, potentially resulting in wrong decisions made by analysts. Such errors can be very detrimental in high-stakes settings such as SOCs, where a data breach or system compromise may occur. This proves that prompt engineering is not merely a technical ability, but also a task of security concern. Prompts should be strategically formulated by the analysts to allow the model to concentrate on the pertinent indicators and to generate helpful and precise outcomes.

On the whole, although tools such as log-analyzer.py have high potential in helping with log analysis, they need to be properly incorporated into current workflows and be designed with care in order to be effective. Their application in actual security operations might pose risks instead of mitigating them in the absence of these considerations.

3. Among the key issues with the application of LLMs in security analysis is that they tend to generate hallucinated outputs, i.e. the model produces information that is not found in the data. When it comes to a security environment, taking such erroneous actions may cause grave repercussions. As an illustration, a malicious activity/an analyst may label normal system behavior as malicious leading to unprocedural blocking of users or service termination. What is more important is that a perceived safe conclusion may trigger a real attack to be overlooked and it might go unnoticed.

In order to minimize these risks, analysts must never blindly trust the output of LLM. Rather, the outcomes of LLM are to be considered starting points that lead to further research. It is necessary to cross-check the results with the help of log queries, SIEM tools, or by hand. Moreover, well-formulated prompts and the narrowness of analysis can also be used to minimize the risk of hallucination. All in all, there should be a balance between trust and careful validation of LLMs.

4. The local LLM is particularly significant in the context of security log analysis since logs can hold sensitive data about operations. These are internal IP addresses, logins, user names and system level events. When this kind of data is uploaded to a cloud-based LLM service, it brings about the risks of data privacy and security.

The data that is stored or processed by cloud-based APIs can be done on the outside, meaning that organizations lack the full control of the way their data is processed. This may lead to compliance problems particularly in

sectors where the data protection is very stringent. Also, sensitive infrastructure information may be revealed in case an attacker gets access to this external system.

Local models, on the other hand, do not outsource the data to the outside environment of the organization. This minimizes the risk of information leakage and gives more control of information processing. In applications that are security sensitive such as log analysis, it is important to ensure that the data is kept confidential and therefore local deployment would be safer.

5. According to this task, it is possible to consider that LLM-based log analysis is more of a supplementary technology, but not a substitute to the old-fashioned technology. Although, LLMs are highly effective to recognize patterns and summarize complicated data, they are not always consistent and accurate.

The classical methods like SIEM systems and regex-based analysis are also needed since they do give precise and accurate results. On the other hand, LLLMs are more interpretation-oriented and can make ambiguous or false assumptions. This renders them inappropriate when it comes to making critical security decisions independently.

Nevertheless, the LLM can greatly enhance efficiency, as it can assist analysts to locate areas of interest in large datasets more promptly. They may help cut down on the time required to initially analyse and give meaningful context. Thus, the combination of the two methods is the most feasible. The LLMs are capable of helping analysts to interpret the data, whereas conventional tools guarantee accuracy and correctness. This moderated strategy is most successful in actual security activities.